

Day 7 — Assignment

Date-Time Data — extended practice

Sections 1–3 use the plain datetime module; Section 4 brings in pandas; Section 5 is a set of standalone questions; Section 6 applies everything to a real dataset, `bike_share_data.csv`.

Section 1 — Datetime Module Basics

Task 1: Create a date object for March 15th, 2024 using `date()`. Also create the string `'2024-03-15'`. Print both and check their types — confirm they look identical but are NOT the same type.

Task 2: Get today's date with `date.today()`. Print the day, month, and year separately.

Task 3: Create a time object for 9:15:45 AM with 250 microseconds. Print the hour, minute, second, and microsecond separately.

Task 4: Combine a date and a time into one datetime object for March 15th, 2024 at 9:15:45 AM, using the `datetime()` constructor directly.

Task 5: Get the current moment with `datetime.now()`. Print both `.weekday()` and `.isoweekday()` for it, and note which day of the week that actually is.

Task 6: Using that same current moment, find which numbered week of the year it falls in.

Section 2 — `strptime` and `strftime`

Task 7: Parse the string `'15 March, 2024 09:30:00'` into a real datetime object using `strptime()`.

Task 8: Parse the string `'2024-03-15'` into a datetime object two ways: once as a full datetime, and once chained with `.date()` to get just the date portion. Print both results and their types.

Task 9: Take the current datetime and format it into a string that reads like `'Sunday, June 28, 2026 at 11:04 AM'` — full weekday name, full month name, day, year, and 12-hour time with AM/PM.

Section 3 — `Timedelta` Calculations

Task 10: Create two datetime objects: January 1st, 2023 at 8:00 AM, and June 15th, 2024 at 5:30 PM. Subtract the earlier one from the later one and print the resulting timedelta, along with its .days and .seconds attributes.

Task 11: Take that same duration and convert it into a number of hours, and separately into a number of minutes.

Task 12: Starting from March 15th, 2024, calculate the date 10 days later and the date 3 weeks earlier.

Section 4 — Pandas Datetime, date_range, and the .dt Accessor

Task 13: Use pd.to_datetime() to parse the string '3rd 9 2024'. What type comes back, and does pandas raise any warning?

Task 14: Use pd.date_range() to generate every day in September 2024 (from 2024-09-01 to 2024-09-30). How many dates are in the result?

Task 15: Use pd.date_range() again, this time starting at 2024-09-01 and asking for 14 periods instead of an end date.

Task 16: Build a small DataFrame with three columns: Order_date (8 consecutive days starting 2024-01-01), Ship_date (8 weekly dates starting 2024-01-01), and Priority (8 random integers from 1 to 3). Then add two more columns using the .dt accessor: Order_month (the month number of Order_date) and Ship_day_of_week (the day name of Ship_date).

Section 5 — Standalone Practice Questions

Task 17: Write a function that checks whether a year is a leap year, and test it on 1996, 2100, 2400, and 2023.

Task 18: Convert the string 'Dec 25 2023 6:15 AM' into a real datetime object.

Task 19: Starting from February 28th, 2020, find the date exactly 10 days later.

Task 20: Convert the Unix timestamp 1700000000 into a readable local datetime. Then separately, print yesterday's date, today's date, and tomorrow's date.

Section 6 — Real Dataset Application

Task 21: Read bike_share_data.csv. Check the dtype of the Start Date and End Date columns — are they real datetime columns, or plain text?

Task 22: Convert both Start Date and End Date to real datetime columns using pd.to_datetime(), and confirm the dtype change with info().

Task 23: Using the .dt accessor on your newly-converted Start Date column, create two new columns: start_hour (the hour of day the trip started) and start_day_of_week (the

full day name). Then use `value_counts()` on `start_day_of_week` to see which day has the most trips.